SMU Lyle School of Engineering



Reinforcement Learning

Prediction and Control with Approximation

Sutton/Barto* Chapter 9-11

Michael Hahsler, SMU

With figures from Sutton/Barto*

*Sutton and Barto, Reinforcement Learning: An Introduction, 2nd edition, MIT Press, Cambridge, MA, 2018



Online Material

CC BY NC SA

Topics of this Course

- Introduction to reinforcement learning
- Markov decision processes
- Part I: Tabular Methods
  - Dynamic programming
  - Monte Carlo methods
  - Temporal-difference learning
  - Multi-step bootstrapping
  - Planning and learning with tabular methods
- **Part II: Approximate Solution Methods**
  - **Prediction and Control using Approximation**
  - Eligibility Traces
  - Policy Gradient Methods
- Part III: Modern RL Methods
  - Deep Reinforcement Learning
  - Current Applications

Summary of Notation

General

X	capital letters: random variables
x, p	lower-case letters: realizations of random variables or scalar functions
$\boldsymbol{w}$	Bold lower-case letters: real-valued vectors (even if random variables)
$\boldsymbol{W}$	bold capitals: matrices
α	Greek letters: parameters (vectors if in bolt)
$\Pr\{X = x\}$	probability that a random variable X takes on the value x
$X \sim p$	random variable X selected from distribution $p(x) = \Pr\{X = x\}$
$\mathbb{E}[X]$	expectation of a random variable X, i.e., $\mathbb{E}[X] = \sum_x p(x)x$
$\text{argmax}_a f(a)$	a value of action a at which $f(a)$ takes its maximal value

Value Function

G_t	return (cumulative reward) following time t
$G_{t:h}$	return from t to h (discounted and corrected)
$v_\pi(s)$	value of state s under policy π (expected return)
$v_*(s)$	value of state s under the optimal policy
$q_\pi(s, a)$	value of taking action a in state s under policy π
$q_*(s, a)$	value of taking action a in state s under the optimal policy
V, V_t	array estimates of state-value function v_π or v_*
Q, Q_t	array estimates of action-value function q_π or v_*

Temporal Difference Learning

U_t	target for estimate at time t
δ_t	TD error

MDP

s, s'	states
a	an action
r	a reward
$\mathcal{S}$	set of all (nonterminal) states, $\mathcal{S}^+$ are all states
$\mathcal{A}(s)$	set of all actions available in state s
γ	discount-rate parameter
t	discrete time step
T	final time step of an episode (a.k.a. horizon)
A_t	random variable for the action at time t
S_t	random variable for the state at time t
R_t	random variable for the reward at time t
$p(s', r \mid s, a)$	probability of transition to state s' and receiving reward r, from state s taking action a.
$p(s' \mid s, a)$	probability of transition to state s' fom state s taking action a.
$r(s, a)$	expected immediate reward from state s after action a.
$r(s, a, s')$	expected immediate reward from state s to s' with action a.
$\pi(a \mid s)$	probability of taking action a in state s under stochastic policy π
$\pi(s)$	action taken in state s under deterministic policy π

Approximation

$\mu(s)$	on policy distribution (fraction of time spent in state)
α	step size (gradient descent learning rate)
$\hat{v}(s, \boldsymbol{w})$	approximate value function
$\hat{q}(s, a, \boldsymbol{w})$	approximate action-value function
$\pi(a \mid s, \boldsymbol{\theta})$	approximate policy
$\widehat{VE}$	mean squared value error
$\nabla f(x)$	gradient of function f evaluated at x

Approximate Solution Methods

Motivation and Definition

Motivation

Value Function

s	State Value $V(s)$
(1,1)	0.7453
(1,2)	?
(1,3)	0.8016
…	…

Issue: Tabular methods do not scale for large state spaces.

a. **Space complexity**: Memory needed for storing the value function with $O(|\mathcal{S}|)$ entries as a table.
b. **No generalization**: State values are stored independently. Many states will never be encountered during learning and therefore cannot be predicted.

Idea: Try to find an **approximate value function** for policy π:

$$\hat{v}(s, \boldsymbol{w}) \approx v_\pi \quad \text{with} \quad \boldsymbol{w} \in \mathbb{R}^d$$

Requirements:

- The function needs to have a **manageable number of parameters,** the weight vector $\boldsymbol{w}$. I.e., $d \ll |\mathcal{S}|$.
- The function can only depend on **features of state** s.
- The function needs to **generalize**, so states with similar features get a similar value.

Approximation functions do this automatically, since all states share a small weight vector

This implies that we cannot change individual state values, as we can in tabular methods! This may be a problem for **greedy policy improvement**!

Goals of Generalization in RL

- In ML: **Can we make predictions for new data?**
  *"Generalization in machine learning is a model's ability to accurately predict outcomes or perform tasks on **new, unseen data**, having learned patterns only from a fixed training dataset."*

- In RL we consider additional goals of generalization:
  a) **How fast does the algorithm learn?** Does it perform well in an environment after gathering limited data (episodes)? (= **sample efficiency**)

  b) **Can the algorithm adapt to changes in the environment?** Does it generalize to a new but related environment? (~ **transfer learning**)

Prediction: Value Function Approximation

Approach: Use **function approximation** (supervised learning).

- We need **training data** in the form of

$$s \longmapsto u,$$

where the target u represents a noisy sample of the state value.

Some options for training data $\mathrm{S_t} \longmapsto U$:

- MC: $\mathrm{S_t} \longmapsto G_t$
- TD(0): $\mathrm{S_t} \longmapsto R_{t+1} + \gamma \hat{v}(S_{\mathrm{t+1}}, \boldsymbol{w}_t)$
- DP: $\mathrm{S_t} \longmapsto \mathbb{E}_\pi[R_{t+1} + \gamma \hat{v}(S_{\mathrm{t+1}}, \boldsymbol{w}_t)]$

Note: The subscript t for $\boldsymbol{w}$ indicates when the update happens during learning. The subscript is often dropped (we only keep track of the current $\boldsymbol{w}$)

Prediction Objective ($\overline{VE}$)

- **Learning algorithm**: Update the weights $\boldsymbol{w}$ to reduce the error for $\hat{v}(s, \boldsymbol{w}) \approx v_\pi$
- **Prediction error** for state s: $|v_\pi(s) - \hat{v}(s, \boldsymbol{w})|$

- To calculate the error over all states we need to define how much we care about each state. We use a state distribution function

$$\mu(s) \geq 0; \quad \sum_s \mu(s) = 1$$

- $\mu(s)$ is typically the fraction of time spent in s when following π.
  We obtain data by following a policy π so samples already comes from this distribution.
  - For continuous tasks, $\mu(s)$ is a stationary distribution called the on-policy distribution.
  - For episodic tasks a near stationary distribution only exists for ergodic* MDPs with large horizons.

Definition: Mean square value error

$$\overline{VE} \stackrel{\text{def}}{=} \sum_{s \in \mathcal{S}} \mu(s)[v_\pi(s) - \hat{v}(s, \boldsymbol{w})]^2$$

Remember: Squared errors have useful mathematical and statistical properties that make learning and optimization easier and more effective.

*Ergodic: The Markov process when following π is irreducible (all states are reachable) and aperiodic (does not get stuck).

Learning Goal

- Find the parameter vector $\boldsymbol{w}^*$ that leads to the smallest error (the global optimum):

$$\overline{VE}(\boldsymbol{w}^*) \leq \overline{VE}(\boldsymbol{w}) \qquad \forall\, \boldsymbol{w}$$

- **Local optima**: Often, we will only find a local optimum where the equation above only holds for $\boldsymbol{w}$ in a small neighborhood around $\boldsymbol{w}^*$.

  Note: Policies converge typically way before the value function converges, so finding the global optimal value function may not be necessary.

- **Issues of interest:**
  - **Convergence**: Convergence to the global or a local optimum depends on the used approximation and the learning method. We will discuss this for each method.
  - **Bias**: Different approximation methods have different bias (=are not able to represent the true value function).

- We still need:
  1. A method to find $\boldsymbol{w}$ for the global or a local optimum.
  2. Define the approximation function $\hat{v}(s, \boldsymbol{w})$.

Gradient Methods for Prediction

How to find $\boldsymbol{w}$ for the global or a local optimum

Gradient Learning Method

Setup

- For a given policy π, learn an approximation $\hat{v}(s, \boldsymbol{w}) \approx v_\pi$
- We use $\boldsymbol{w} \stackrel{\text{def}}{=} (w_1, w_2, \ldots, w_d)$
- Choose $\hat{v}(s, \boldsymbol{w})$ as a **differentiable function** of $\boldsymbol{w}$ for all $s \in \mathcal{S}$ so it has a gradient

Learning

- **Sample following $\boldsymbol{\pi}$** and observe at each discrete time step $t = 0, 1, 2, 3, \ldots$ a new example

$$S_t \mapsto v_\pi(S_t).$$

For now, we assume that we can assess the correct $v_\pi(S_t)$

- **Update $\boldsymbol{w}$** to reduce $\overline{VE}$. I.e., make $\hat{v}(s, \boldsymbol{w})$ more similar to v_π.

$$\overline{VE} \stackrel{\text{def}}{=} \sum_{s \in \mathcal{S}} \mu(s)[v_\pi(s) - \hat{v}(s, \boldsymbol{w})]^2$$

Note that the samples automatically follow the correct state distribution μ for π

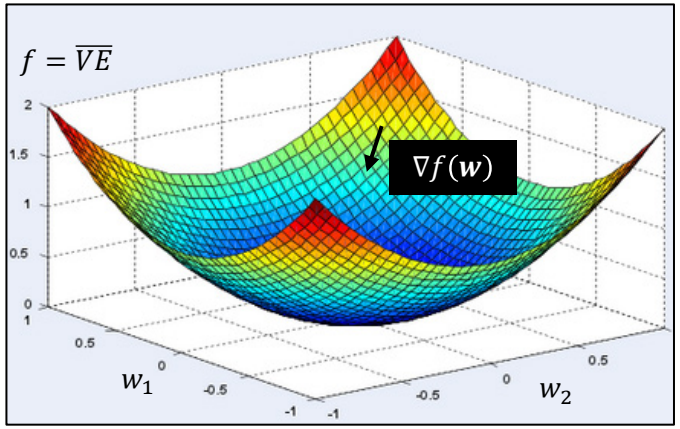
SGD: Stochastic-Gradient Descend

Minimize $\overline{VE} \stackrel{\text{def}}{=} \sum_{s \in \mathcal{S}} \mu(s)[v_\pi - \hat{v}(s, \boldsymbol{w})]^2$ by moving $\boldsymbol{w}$ in the direction $\nabla \overline{VE}$.

Gradient: $\nabla f(\boldsymbol{w}) = \left(\frac{\partial f(\boldsymbol{w})}{\partial w_1}, \frac{\partial f(\boldsymbol{w})}{\partial w_2}, \ldots, \frac{\partial f(\boldsymbol{w})}{\partial w_d}\right)^\top$

Update for one observation: $S_t \mapsto v_\pi(S_t)$

$$\begin{aligned} \boldsymbol{w}_{t+1} &= \boldsymbol{w}_t - \frac{1}{2}\alpha \, \nabla [v_\pi(S_t) - \hat{v}(S_t, \boldsymbol{w}_t)]^2 \\ &= \boldsymbol{w}_t + \alpha [v_\pi(S_t) - \hat{v}(S_t, \boldsymbol{w}_t)] \, \nabla \hat{v}(S_t, \boldsymbol{w}_t) \end{aligned}$$



Applying the chain rule* shows that we only need the gradient of the approximation function.
Note: the sign changes and $\frac{1}{2}$ was chosen for convenience.

- **Important property of SGD**: Only depends on the gradient of the approximation function. Does not need the gradient of the unknown value function!
- **Stochastic**: because update is on a single (randomly chosen) example for S_t.
- **Step size α**: Since we update towards a sampled value (sample variance!) and updates of $\boldsymbol{w}$ for one state will change the values for other states as well, we typically use many small updates (= a small value of α).

***Chain rule** (of differentiation)

$$h(x) = z(y(x))$$

is equivalent to

$$h'(x) = z'(y(x))\, y'(x)$$

Here x is the vector $\boldsymbol{w}$ and we have partial derivatives written as gradients.

Step Size for SGD

- SGD converges to a **local optimum** if the **stochastic approximation criterion** (Robbins–Monro conditions) is met:

$$\sum_{t=1}^{\infty} \alpha_t(s,a) = \infty \quad \text{and} \quad \sum_{t=1}^{\infty} \alpha_t^2(s,a) \leq \infty$$

$\alpha_t(s,a)$ … step size used for action a in state s at step t.

- **Popular choices for α:**
  - Tabular MC uses sample averaging which is equivalent to $\alpha_t(s,a) = \frac{1}{N_t(s,a)}$ with $N_t(s,a)$ is how often the action was chosen in that state. **Not appropriate for TD or approximation**!
  - Constant $\alpha = 1/\tau$ means a tabular estimate would approach the mean target after τ experiences + keeps learning.

- **How to choose α in practice**
  - Typical values: $0.1, 0.01, 0.001$
  - If learning diverges: reduce α by a factor of 10
  - If learning is too slow: increase α bey a factor of 2 to 5.
  - Plot learning curve.

A constant does not satisfy the stochastic approx. criterion.

Using a Target Estimate for SGD

Issue: We have assumed that we know the actual value of $v_\pi(S_t)$ for the update

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t + \alpha[v_\pi(S_t) - \hat{v}(S_t, \boldsymbol{w}_t)] \nabla \hat{v}(S_t, \boldsymbol{w}_t)$$

but that is not the case in RL where we want to learn v_π.

Solution: We can use any target estimate of the expected return U_t instead

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t + \alpha[U_t - \hat{v}(S_t, \boldsymbol{w}_t)] \nabla \hat{v}(S_t, \boldsymbol{w}_t)$$

SGD converges to a **local optimum** as long as U_t is an

- **unbiased estimate**. I.e., $\mathbb{E}(U_t | S_t = s) = v_\pi(s)$
- we use decreasing α
- local gradient domination holds (e.g., linear approximation)

Examples:

- The Gradient Monte Carlo method uses the unbiased MC estimate $U_t = G_t$

Gradient Monte Carlo Algorithm

Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \to \mathbb{R}$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, S_1, A_1, \ldots, R_T, S_T$ using π
  Loop for each step of episode, $t = 0, 1, \ldots, T-1$:
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[G_t - \hat{v}(S_t, \mathbf{w})\big]\nabla\hat{v}(S_t, \mathbf{w})$

Remember from MC:

$$G_t = \sum_{k=0}^{T} \gamma^k R_{t+k+1}$$

Bootstrapping and Semi-Gradient Methods

Issue: To get the true gradient requires, the target has to be **unbiased**, fixed and only the prediction can depend on $\boldsymbol{w}$.

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t + \alpha[U_t - \hat{v}(S_t, \boldsymbol{w}_t)] \nabla \hat{v}(S_t, \boldsymbol{w}_t)$$

Gradient Monte Carlo Algorithm: Works since $U_t = G_t$ is the result of only sampled rewards which are independent of $\boldsymbol{w}$.

Bootstrapping (e.g., n-step TD): breaks this requirement because U_t depends on the bootstrapped estimate $\hat{v}(S_{t+n}, \boldsymbol{w})$ and thus the current value of $\boldsymbol{w}$.
This does not result in a true gradient, and convergence cannot be guaranteed.

Semi-Gradient Methods:

- If we a biased target estimate, then this is called a semi-gradient method.
- Advantages:
  - Usually learn faster.
  - Often still converge (especially for linear approximators).
  - TD can learn online.

Semi-gradient TD(0)

Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \to \mathbb{R}$ such that $\hat{v}(\text{terminal},\cdot) = 0$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose $A \sim \pi(\cdot|S)$
    Take action A, observe R, S'
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[R + \gamma\hat{v}(S',\mathbf{w}) - \hat{v}(S,\mathbf{w})\big]\nabla\hat{v}(S,\mathbf{w})$
    $S \leftarrow S'$
  until S is terminal

U_t is a bootstrapped estimate and biased because it uses $\boldsymbol{w}$. Convergence is not guaranteed!

Linear Approximation Methods

Definition of a simple approximation function $\hat{v}(s, \boldsymbol{w})$

Linear Methods

- Linear approximation of the state value by the inner product

$$\hat{v}(s, \boldsymbol{w}) \stackrel{\text{def}}{=} \boldsymbol{w}^\top \boldsymbol{x}(s) = \sum_{i=1}^{d} w_i x_i(s)$$

$\boldsymbol{x}(s)$… a state feature vector with $x_i : \mathcal{S} \to \mathbb{R}$ (called basis function)

- **SGD gradient**: $\nabla \hat{v}(s, \boldsymbol{w}) = \left(\frac{\partial \hat{v}(s,\boldsymbol{w})}{\partial w_1}, \frac{\partial \hat{v}(s,\boldsymbol{w})}{\partial w_2}, \dots, \frac{\partial \hat{v}(s,\boldsymbol{w})}{\partial w_d} \right)^\top = (x_1, x_2, \dots, x_d)^\top = \boldsymbol{x}(s)^\top$
- **SGD update**: $\boldsymbol{w}_{t+1} = \boldsymbol{w}_t + \alpha[U_t - \hat{v}(S_t, \boldsymbol{w}_t)]\boldsymbol{x}(S_t)$

These are extremely strong guarantees!

- **Advantages**:
  - Simple math.
  - Has only one optimum and is **guaranteed to converge** to the **global minimum** of $\overline{VE}$ if α is reduced following the stochastic approximation condition.
  - Convergence guarantee also holds for **bootstrapping** methods like Semi-gradient TD(0)!

Feature Vector

- **Issue**: How do we construct the state feature vector $\boldsymbol{x}(s)$?

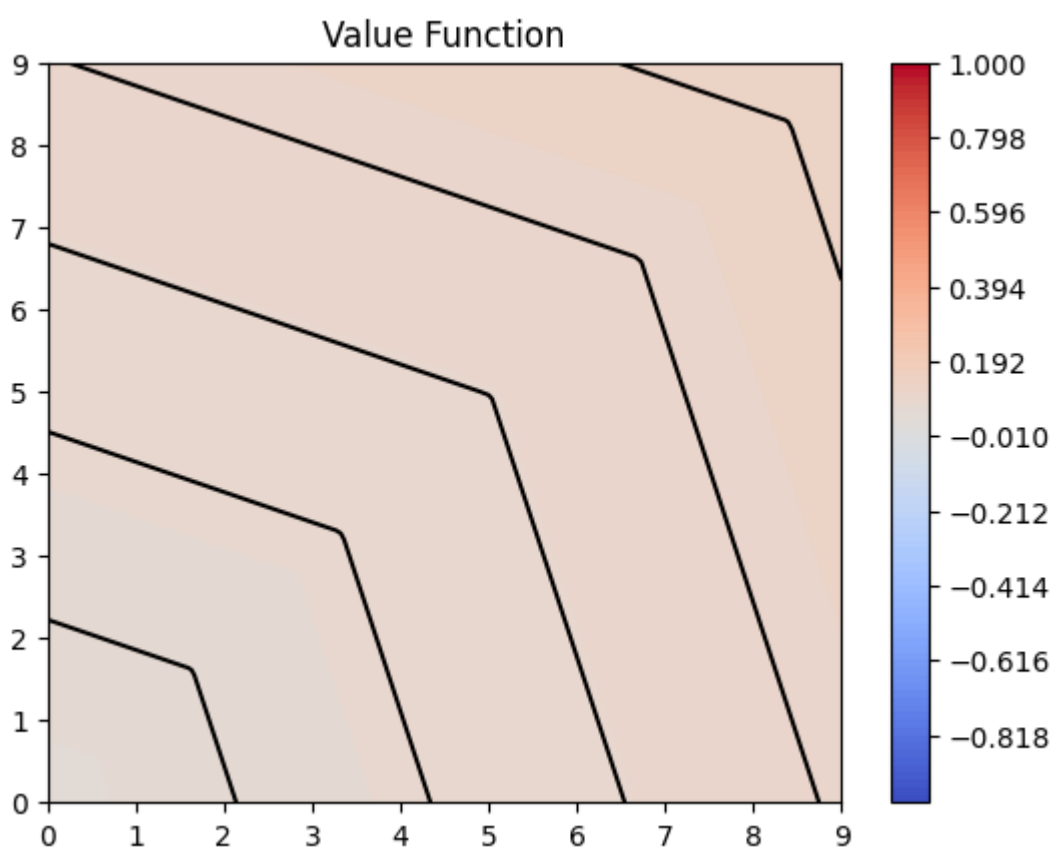
- **Initial idea**: If the factored state description consists of several numbers (e.g., x and y coordinates), then we can directly use them as the feature vector $\boldsymbol{x}(s) = (s_x, s_y)$.

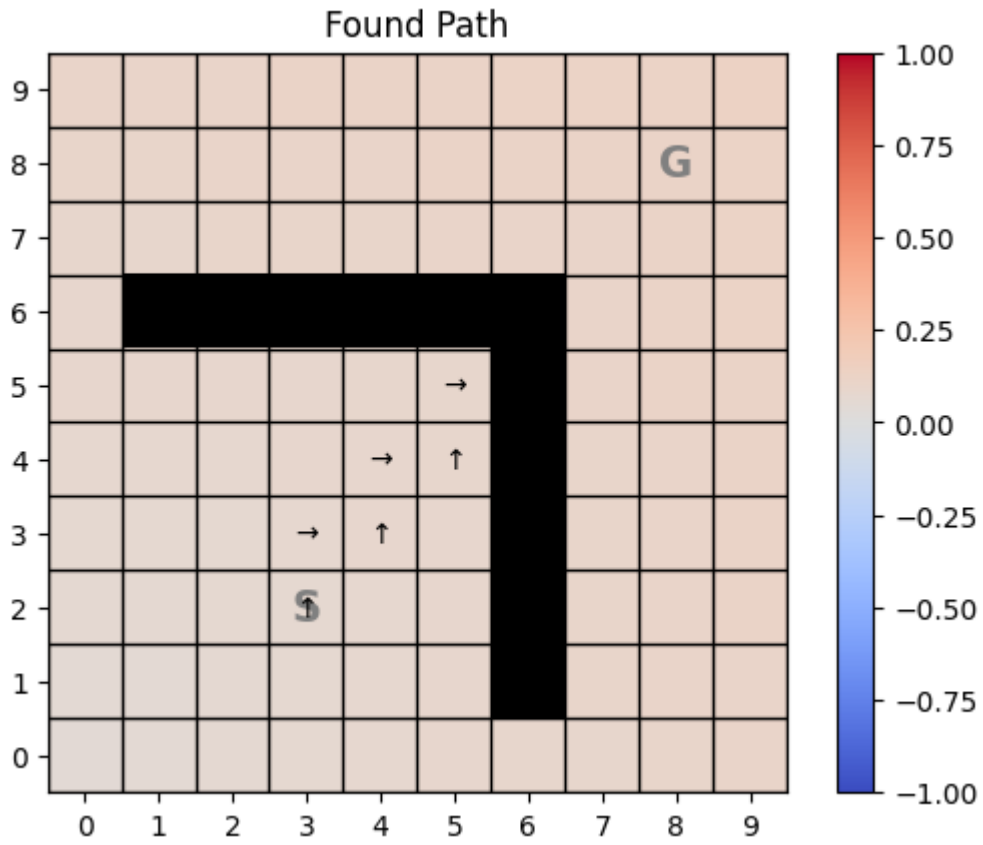
  This leads to **linear interpolation** learned as a **linear regression.**

- **Problems**:
  - The value function may not depend linearly on the variables used in the factored state description.
  - Linear models cannot represent interactions between variables.

Example: L-Maze with Linear Features

Semi-gradient Sarsa (see Control later) with linear features cannot solve the L-Maze





Feature Construction

- **Idea**: Use a more flexible, non-linear transform ϕ of the state description to construct state features.

$$\boldsymbol{x}(s) = \phi(s)$$

- We will focus on the most popular feature construction methods that work well for RL:
  - **Fourier Basis:** strong baseline; very efficient; strong guarantees.
  - **Tile Coding**: coarse coding using tiles, local generalization, very computationally efficient.

- Other methods
  - One-Hot Features: exact representation; no generalization (equivalent to tabular RL)
  - Coarse Coding using spherical receptor fields.
  - Polynomial Features: smooth global generalization.
  - Radial Basis Functions: smooth local generalization.

Fourier Basis

- A **Fourier series** (Fourier transform) represents a periodic function as a weighted sum of sine and cosine basis functions of increasing frequencies.
- State features are not periodic, so we consider only a single period, chosen so that we only need to use cosine basis functions.
- **On dimensional case**: state is represented by a single number $s \in [0,1]$

$$x_i(s) = \cos(i\pi s)$$

π is 180° in radians

with $i = 0, \dots, n$ where i is the frequency and n is the order of the Fourier basis function

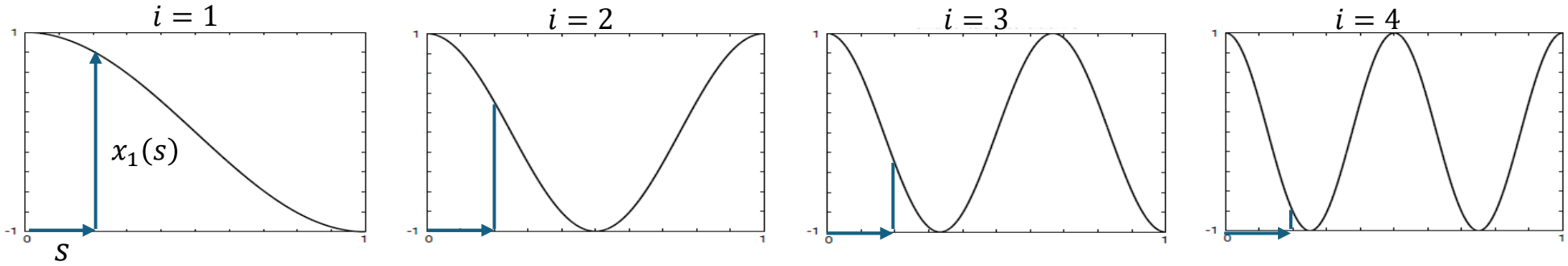
- Strong theoretical foundation: Fourier series can approximate any function.

Example: Fourier Transform



Source: Wikipedia

Example: Order-4 Fourier cosine basis creates 4 features plus the intercept for $i = 0$ for linear approximation.



$$\boldsymbol{x}(.2) = (1, .9, .65, .35, .1)$$

Multi-dimensional Fourier Basis

- State is represented by the normalized features as vector $\boldsymbol{s} = (s_1, s_2, \ldots, s_k) \in [0,1]^k$

$$x_i(\boldsymbol{s}) = \cos(\pi \boldsymbol{s}^\top \boldsymbol{c}^i)$$

where $\boldsymbol{c}^i = (c_1^i, \ldots, c_k^i)$ is a coefficient vector with $c_j^i \in \{0, \ldots, n\}$ for $j = 1, \ldots, k$ and $i = 1, \ldots, (n+1)^k$

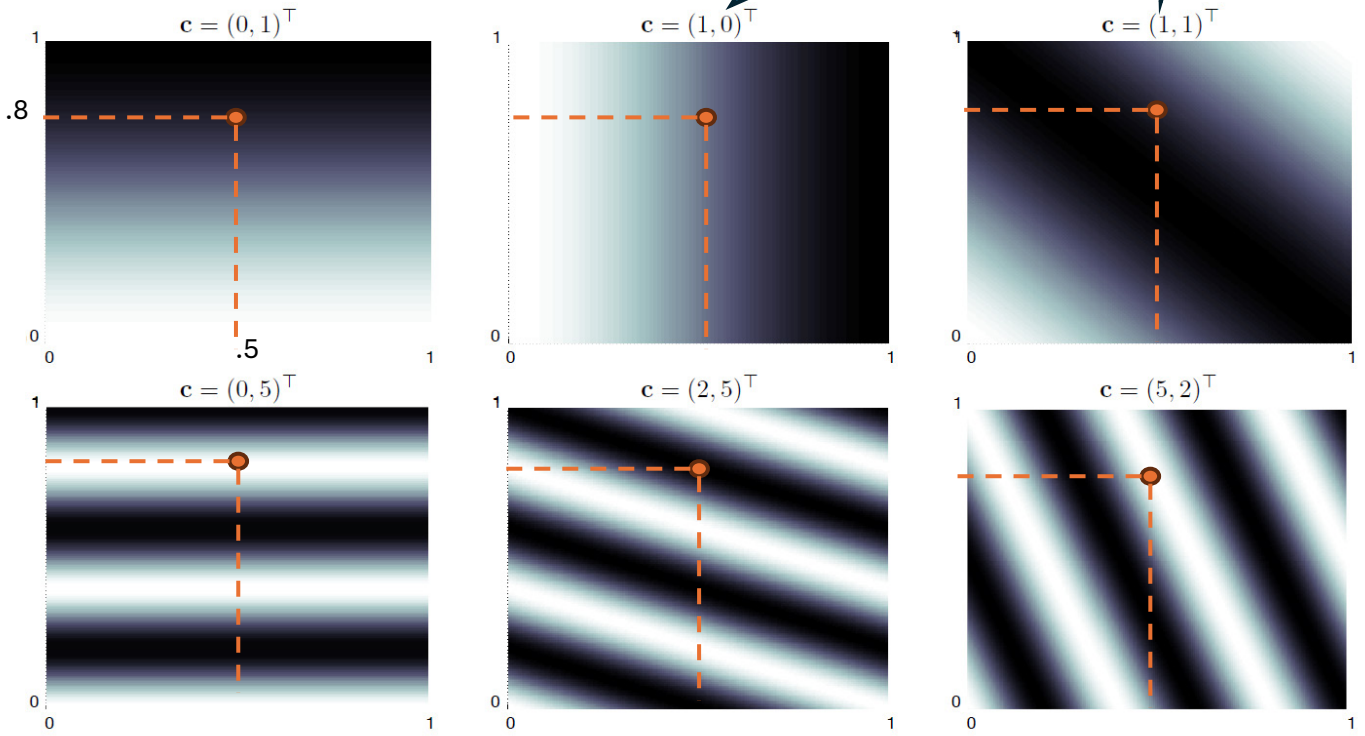
Example: 2-dimensional Fourier cosine features for 6 different $\boldsymbol{c}$.

$$\boldsymbol{s} = (.5, .8)$$

$x_0(\boldsymbol{s}) = 1$
$x_1(\boldsymbol{s}) = \cos(\pi\ (.5, .8)^\top (0,1)) = -.8$
$x_2(\boldsymbol{s}) = \cos(\pi\ (.5, .8)^\top (1,0)) = 0$
$x_3(\boldsymbol{s}) = \cos(\pi\ (.5, .8)^\top (1,1)) = -.6$
$x_4(\boldsymbol{s}) = \cos(\pi\ (.5, .8)^\top (0,5)) = 1$
$x_5(\boldsymbol{s}) = \cos(\pi\ (.5, .8)^\top (2,5)) = -1$
$x_6(\boldsymbol{s}) = \cos(\pi\ (.5, .8)^\top (5,2)) = 1$

>0 is the frequency

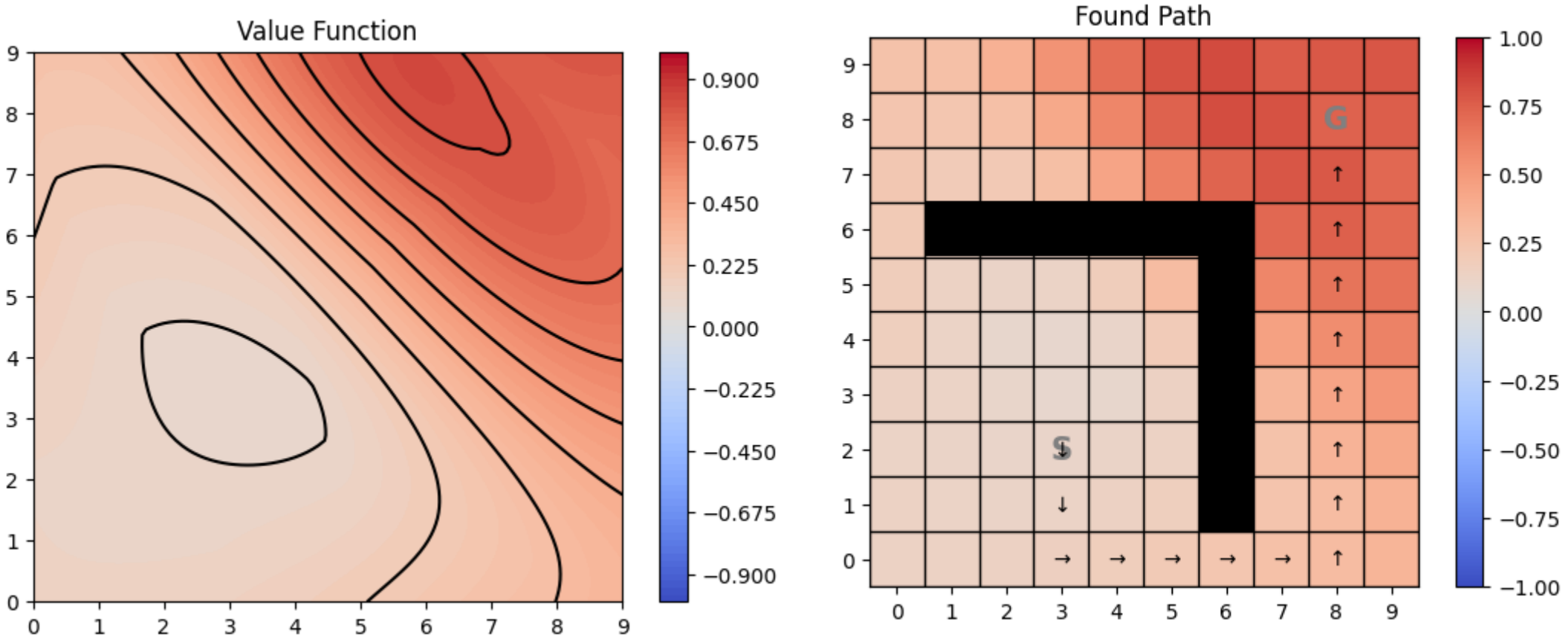
0 means it is constant in that dimension



1. Features are a combination of both state components, representing interactions.
2. Select only the best features.
3. We can use a different α for each feature.

Example: L-Maze with Order-2 Fourier Features

Semi-gradient Sarsa with order-2 Fourier Features can solve the L-Maze



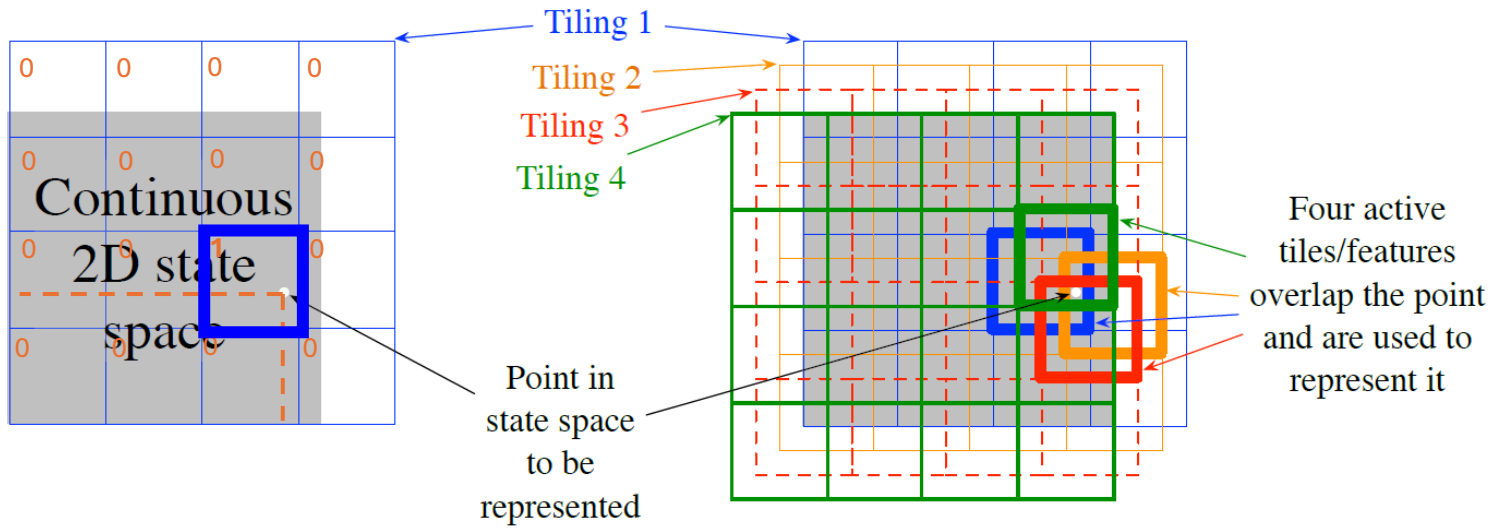
Tile Coding

- Partition the space using several **overlapping grids**.
- Represent the state by the **"active" tiles** in the grids as a vector $\boldsymbol{x}$ with 1 for all active tiles and 0 for all others.
- **Similar states will share active tiles**. Leads to a local generalization.
- α is typically chosen to be small since a change will also affect neighboring states.

  Popular: $\alpha = \frac{1}{10\,\#\text{tilings}}$

Example:

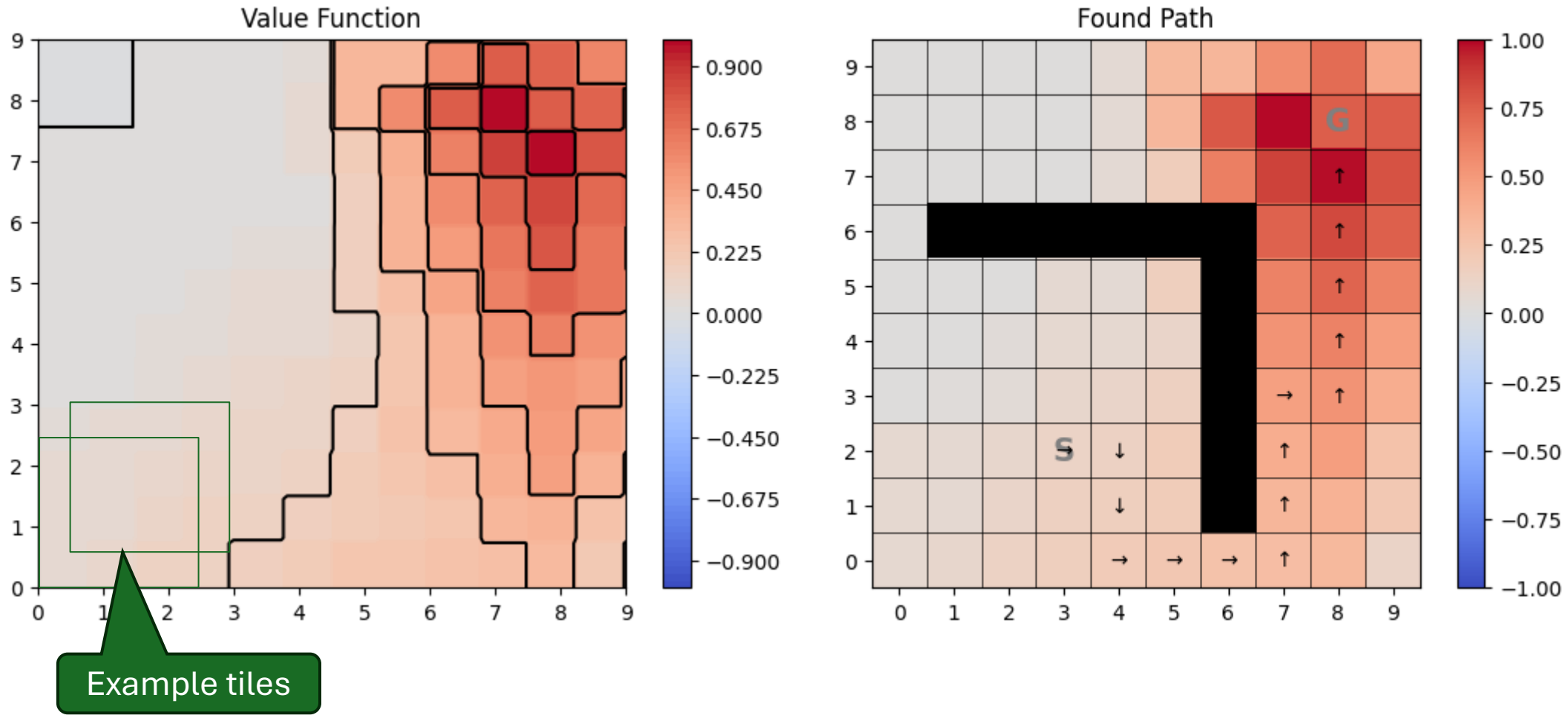
- State has two components.
- 4 4×4 overlapping tilings give 64 squares.
- State features: 60 0s and only the four active tiles have a 1



- **Issue**: High-dimensional state spaces lead to too long sparse feature vectors. Idea: Use index hash tables with limited capacity to focus on more frequently encountered states.

Example: L-Maze with Tile Coding

Semi-gradient Sarsa with Tile Coding (4 tilings, 4 tiles per dimension) can solve the L-Maze



Tabular RL as a Special Case

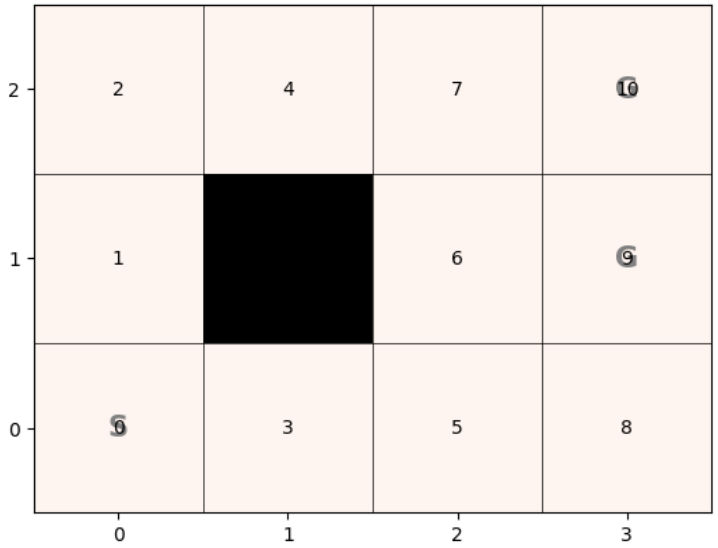
Tabular RL is a special case of Linear Approximation for a discrete state space.

- **State feature vector**: $\boldsymbol{x}$ is a one-hot encoding for the state.
- **Weight vector**: $\boldsymbol{w}$ has 1 entry for each state.

- $\hat{v}(s, \boldsymbol{w}) = \boldsymbol{w}^\top \boldsymbol{x}(s)$ now always picks a single value from $\boldsymbol{w}$.

- Consequences:
  - $\boldsymbol{w}$ is just V in our tabular methods.
  - No generalization

Example:



$\boldsymbol{x}(s(0,0)) = (1,0,0,0,0,0,0,0,0,0,0)$
$\boldsymbol{w} = (V_0, V_1, V_2, \dots, V_{10})$

$\hat{v}(s(0,0), \boldsymbol{w}) = \boldsymbol{w}^\top \boldsymbol{x}(s(0,0)) = V_0$

Practical Issue: Partially Observable Environments

- Observations may be incomplete or noisy.

- Standard model**: POMDP** with belief states
  - A key feature is **state estimation**: It learns **hidden states** depending on the history of actions and observations.
  - **Impractical** due to the size of the belief state space.

- Practical approach for RL with approximation
  1. Build a **compressed representation** with features from current observations and the observation history (E.g., observation stacking).
  2. **Use a (non) linear approximator** with those features to learn a value function or policy.

- **Example**: Maze solver where the x and y coordinates are noisy. Using the observed coordinates from the last few steps (stacking them) helps average out random noise in the observations.

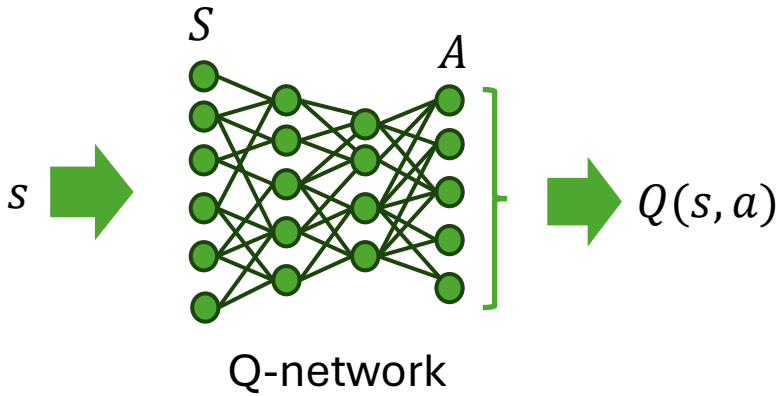
Non-Linear Function Approximation

Artificial Neural Networks to approximate $\hat{v}(s, \boldsymbol{w})$

Approximation with ANNs

- Uses simple networks with a single hidden layer or deep architectures to approximate the value function.
- Architecture choices:
  - Single hidden-layer feedforward network (is a universal approximator)
  - Convolutional networks
  - Recurrent networks
  - Transformers
  - Deep residual learning…

  + Regularization, dropout, weight sharing, …



- Training: Use SGD and backpropagation for deep networks.

- Issues due to nonlinearity:
  - Instability
  - Generally, no convergence guarantee.

- More in Deep Reinforcement Learning (DRL).

On-Policy Control with Approximation

Semi-Gradient Sarsa

Control Problem

- Estimate a parametric approximation for the value-action function.

$$\hat{q}(s, a, \boldsymbol{w}) \approx q_*(s, a)$$

- **Training data**: $S_t, A_t \mapsto U_t$
- **Update**: $\boldsymbol{w}_{t+1} = \boldsymbol{w}_t + \alpha[U_t - \hat{q}(S_t, A_t, \boldsymbol{w}_t)] \nabla \hat{q}(S_t, A_t, \boldsymbol{w}_t)$
  - U_t can be any target estimate:
    - MC: $U_t = G_t$
    - 1-step TD: $U_t = R_{t+1} + \gamma \hat{q}(S_t, A_t, \boldsymbol{w}_t)$
    - …

A biased estimate does not give us the true gradient! This will lead to a **semi-gradient method**.

- **Policy improvement**: update policy with greedy action

$$A^*_{t+1} = \underset{a}{\operatorname{argmax}}\, \hat{q}(S_{t+1}, a, \boldsymbol{w}_t)$$

The update of $\boldsymbol{w}$ may have changed $\hat{q}$ for several states and actions! May be an issue with improvement.

- **Exploration**: For on-policy control, use a soft approximation to the greedy policy (e.g., ϵ-greedy).

State-Action Features

Issue: We need to learn to predict $\hat{q}(s, a, \boldsymbol{w})$, which should have a different value for each action.

Geramifard et al. (2013)* suggest estimating values for each action separately:

- Different state-feature weights for each action.
- For prediction/update, only the intercept and the weights belonging to the action are used.

Example: States described by x and y coordinates + 3 action

- Weight vector

$$\boldsymbol{w} = (\mathrm{w}_{\mathrm{intercept}}, \mathrm{w}_{\mathrm{x,action1}}, \mathrm{w}_{\mathrm{y,action1}}, \mathrm{w}_{\mathrm{x,action2}}, \mathrm{w}_{\mathrm{y,action2}}, \mathrm{w}_{\mathrm{x,action3}}, \mathrm{w}_{\mathrm{y,action3}})$$

- Example state-action feature vector for state (1,3) and action 2:

$$\boldsymbol{x}\big(s(x = 1, y = 3, \mathrm{action2})\big) = (1,0,0,1,3,0,0)$$

*Alborz Geramifard, Stefanie Tellex, and Thomas J. Walsh. A Tutorial on Linear Function Approximators for Dynamic Programming and Reinforcement Learning, Foundations and Trends in Machine Learning, Vol. 6, No. 4 (2013) 375–454

Episodic Semi-gradient Sarsa

Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \to \mathbb{R}$ such that $\hat{v}(\text{terminal},\cdot) = 0$
Algorithm parameter: step size α
Initialize value-function weights $\mathbf{w}$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose $A \sim \pi(\cdot|S)$
    Take action A, observe R, S'
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[R + \gamma\hat{v}(S',\mathbf{w}) -$
    $S \leftarrow S'$
  until S is terminal

Episodic Semi-gradient Sarsa for Estimating $\hat{q} \approx q_*$

Input: a differentiable action-value function parameterization $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \to \mathbb{R}$
Algorithm parameters: step size $\alpha > 0$, small $\varepsilon > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:
  $S, A \leftarrow$ initial state and action of episode (e.g., ε-greedy)
  Loop for each step of episode:
    Take action A, observe R, S'
    If S' is terminal:
      $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[R - \hat{q}(S, A, \mathbf{w})\big]\nabla\hat{q}(S, A, \mathbf{w})$
      Go to next episode
    Choose A' as a function of $\hat{q}(S', \cdot, \mathbf{w})$ (e.g., ε-greedy)
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[R + \gamma\hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})\big]\nabla\hat{q}(S, A, \mathbf{w})$
    $S \leftarrow S'$
    $A \leftarrow A'$

Gradient is $\boldsymbol{x}(S', A')$ for linear approximation

With features: $\hat{q}(\boldsymbol{x}(S', A'), \boldsymbol{w})$

Considerations for Continuing Tasks

Discounting vs. Average Reward

- Discounted rewards (γ-returns) are used for tabular methods because returns for each state are separately identified and averaged.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$

- With approximation, returns are shared between states and it is often better to focus on the long-run **average reward of a policy** per time step.

$$r(\pi) = \lim_{T \to \infty} \frac{1}{T} \sum_{t=1}^{T} R_t$$

The average reward does not depend on the start state.

Differential Returns

- It is often helpful to use differences between the immediate rewards and the average reward given policy π.

$$\mathrm{G_t} = \sum_{k=1}^{\infty} \big(R_{t+k+1} - r(\pi)\big)$$

- Subtracting $r(\pi)$ makes sure that the infinite sum does not diverge.
- Differential returns lead to differential value functions.





What You should Know

- Approximate the value function using parametrized functions:

$$\hat{v}(s, \boldsymbol{w}) \text{ or } \hat{q}(s, a, \boldsymbol{w})$$

- For linear approximation, we need to create **features** that take non-linearities and the interaction of state components into account. Popular are:
  - Fourier basis features
  - Tile coding

- Use **SGD or semi-gradient descent** to learn $\boldsymbol{w}$ from samples by minimizing $\overline{VE}$. The samples already follow the on-policy distribution $\mu(s)$.

- Issue: **Convergence**
  - Biased target estimates?
  - Linear vs. non-linear approximation.
  - choice of α.